CO2 - AT1 : Analytical problem Solving

CSA0401 - Operating systems

(28)

G. Yashmita

192511032

30/07/2026

1. CPU scheduling - Waiting time & Turnaround time analysis.

A set of process arrive at time 0 with the following Burst time.

Process	Burst time
P_1	10
P_2	4
P_3	6
P_4	2

a) Calculate waiting time and Turnaround time for each process using SJF (Non-preemptive)

b) Compute average waiting time & average turnaround time.

c) Interept whetha SJF improves CPU utilization compared to FCFS.

2. Round - Robin Scheduling - Time quantum Impact

Process	Burst time
P_1	20
P_2	5
P_3	13
P_4	8

Let the time quantum = 4 ms.

a) Construct Gantt chart for Round Robin

b) Calculate waiting time and Turnaround Time for each process.

c) Analyze how changing the quantum to 10 ms would affect the system performance (CPU utilization & context switching)

3. Deadlock - Safety check using Banker's algorithm

A system has 3 resource types (A, B, C)

Total resources : A = 10 , B = 5, C = 7

The allocation and maximum Need matrices are :

Allocation Matrix :

Process	A	B	C
P_1	0	1	0
P_2	2	0	0
P_3	3	0	2
P_4	2	1	1
P_5	0	0	2

Maximum Matrix

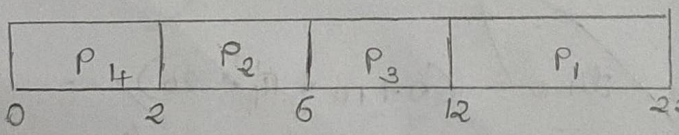
Process	A	B	C
P_1	7	5	3
P_2	3	2	2
P_3	9	0	2
P_4	4	2	2
P_5	5	3	3

a) Calculate the need matrix

b) Determine if the system is in a safe state using Banker's algorithm.

c) provide a valid safe sequence if it exists.

1. a)

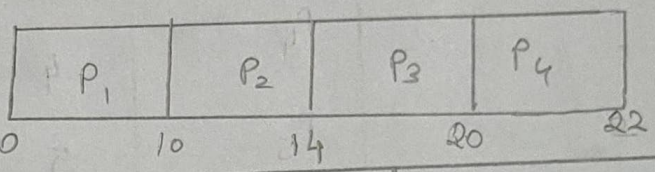


process	Burst time	Turnaround time = CT - AT	Waiting time = TAT - BT
P_1	10	22 - 0 = 22	22 - 10 = 12
P_2	4	6 - 0 = 6	6 - 4 = 2
P_3	6	12 - 0 = 12	12 - 6 = 6
P_4	2	2 - 0 = 2	2 - 2 = 0

b) * Average waiting time = $(12+2+6+0)/4 = 20/4 = 5$

* Average turnaround time = $(22+6+12+2)/4 = 42/4 = 10.5$

c) FCFS - First Come First Serve



Process	Burst time	Turnaround time CT - AT	Waiting time TAT - BT
P_1	10	10 - 0 = 10	10 - 10 = 0
P_2	4	14 - 0 = 14	14 - 4 = 10
P_3	6	20 - 0 = 20	20 - 6 = 14
P_4	2	22 - 0 = 22	22 - 2 = 20

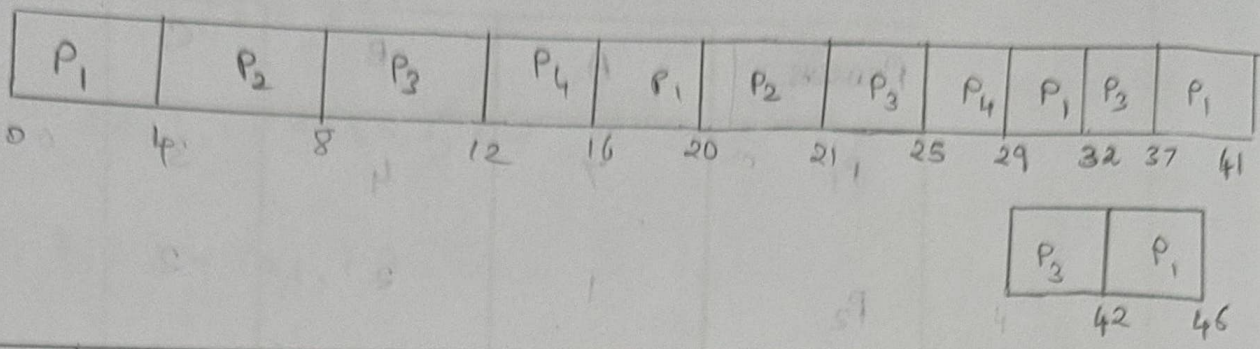
* Average waiting time = $(0+10+14+20)/4 = 44/4 = 11$

* Average turnaround time = $(10+14+20+22)/4 = 66/4 = 16.5$

10

Yes. SJF improves CPU utilization compared to FCFS. Since, SJF average waiting time is less than the FCFS.

2. a) Gantt chart.



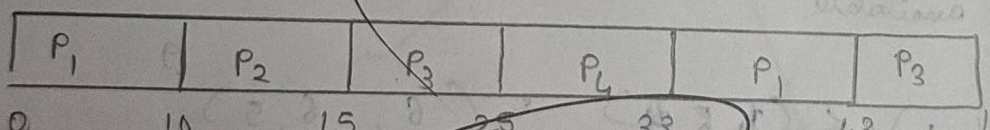
b)

Process	Burst time	Turnaround time	Waiting time
P_1	20	46	26
P_2	5	21	16
P_3	13	42	29
P_4	8	29	21

Avg Waiting time = $(26 + 16 + 29 + 21)/4 = 138/4 = 34.5$

Avg turnaround time = $(46 + 21 + 42 + 29)/4 = 92/4 = 23$

c)



Process	Burst time	TAT	Waiting time
P_1	20	43	23
P_2	5	15	10
P_3	13	46	33
P_4	8	33	25

3. a) Need matrix = Max - Allocation

Available = (3, 3,

Process	A	B	C
P_1	7	4	3
P_2	1	2	2
P_3	6	0	0
P_4	2	1	1
P_5	5	3	1

b) Need $\leq$ Available

$P_1 = (7, 4, 3) \leq (3, 3, 2)$

Not available

$P_2 = (1, 2, 2) \leq (3, 3, 2) = (4, 5, 4)$

Available

P_3 = Not available

$P_4 = (2, 1, 1) \leq (4, 5, 4) = (6, 6, 5)$

Available

$P_5 = (5, 3, 1) \leq (6, 6, 5) = ($

Available

D b) $P_2 = (3,3,2) + (2,0,0)$

$= (5,3,2)$

$(2,1,1) \leq (5,3,2)$

$P_4 = (5,3,2) + (2,1,1)$

$= (7,4,3)$

$P_5 = (5,3,1) \leq (7,4,3)$

$(7,4,3) + (0,0,2)$

$(7,4,3) \leq (7,4,5)$

$P_1 = (7,4,5) + (0,1,6)$

$= (7,5,5)$

$P_3 = (10,5,7)$

c) $P_2 \rightarrow P_4 \rightarrow P_5 \rightarrow P_1 \rightarrow P_3$

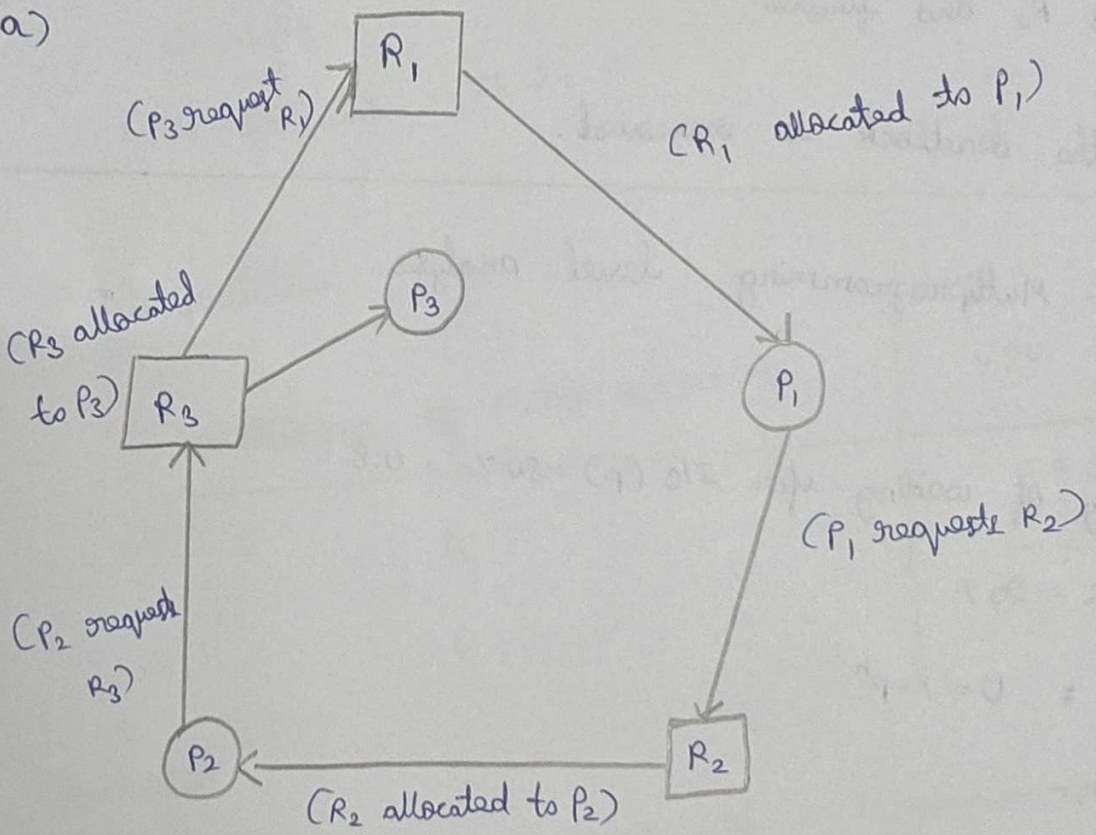
It is in safe state.

4. Deadlock detection - Resource allocation graph (RAG)

Given:

* P_1 is holding R_1 and waiting for R_2
* P_2 is holding R_2 and waiting for R_3
* P_3 is holding R_3 and waiting for R_1

a)



b) $P_1 \rightarrow R_2 \rightarrow P_2 \rightarrow R_3 \rightarrow P_3 \rightarrow R_1 \rightarrow P_1$

Since each resource has one instance and it forms a cycle. The system is in Deadlock.

c) Recommend a strategy:

Recovery Method:

Terminate one of the deadlock process.

After terminating P_3:

* R_3 becomes free
* P_2 gets R_3 and finishes.
* R_2 becomes free.
* P_1 gets R_2 and finishes.

Therefore, the deadlock is removed.

5. CPU utilization - Multiprogramming level analysis.

Given:

Probability of waiting for I/O (p) = 80% = 0.8

CPU time = 20%

Formula : $U = 1 - p^n$

where,

$p = 0.8$

n = Degree of multiprogramming.

a) Degree of multiprogramming = 4

$U = 1 - p^n$

$U = 1 - (0.8)^4$

$[(0.8)^4 = 0.4096]$

$U = 1 - 0.4096$

$U = 0.5904$

$U = 59.04\ \%$

∴ CPU utilization = 59.04 %.

b) Degree of multiprogramming = 6

$U = 1 - (0.8)^6$

$U = 1 - 0.262144$

$U = 0.737856$

$U = 73.79\ \%$

∴ CPU utilization = 73.79 %.

c) Interpretation

Degree of Multiprogramming	CPU utilization
4	59.04 %
6	73.79 %

Conclusion :

* Increasing the degree of multiprogramming from 4 to 6 increases CPU utilization from 59.04 % to 73.79 %.

* And more processes are available to execute while others wait for I/O, reducing CPU idle time.

* Hence, 6 processes provide better system performance.